

BOOK STUDY · CHAPTER-BY-CHAPTER

Software Engineering at Google

— 25장 전체 교습서

Winters · Manshreck · Wright (O'Reilly, 2020) 원서를
전 25장, 장별로 뜯어 가르치는 병행 교재.
시간과 규모가 만드는 문제를
문화·프로세스·도구의 삼각대로 다루는 법.

원서는 구글이 무료 공개함: abseil.io/resources/swe-book

이 문서는 원서의 이해를 돕는 교습서 (원문 복제 아님)

2026년 7월 17일 제작

목차

들어가며: 이 책을 읽는 법

제1부. 논제 (Thesis)

1장. 소프트웨어 엔지니어링이란 무엇인가

제2부. 문화 (Culture)

2장. 팀에서 잘 일하는 법

3장. 지식 공유

4장. 형평성을 위한 엔지니어링

5장. 팀을 이끄는 법

6장. 규모를 갖춰 이끌기

7장. 엔지니어링 생산성 측정

제3부. 프로세스 (Processes)

8장. 스타일 가이드와 규칙

9장. 코드 리뷰

10장. 문서화

11장. 테스트 개관

12장. 단위 테스트

13장. 테스트 더블

14장. 더 큰 테스트

15장. 폐기(Deprecation)

제4부. 도구 (Tools)

16장. 버전 관리와 브랜치 관리

17장. 코드 검색

18장. 빌드 시스템과 빌드 철학

19장. Critique: 구글의 코드 리뷰 도구

20장. 정적 분석

21장. 의존성 관리

22장. 대규모 변경(LSC)

23장. 지속적 통합(CI)

24장. 지속적 배포(CD)

25장. 서비스로서의 컴퓨트

나가며: 이 책의 한 줄

부록. 25장 색인표

들어가며: 이 책을 읽는 법

이 책은 "구글의 코딩 스타일"이 아니다. 수만 명의 엔지니어가 수십억 줄의 코드를 수십 년간 유지하면서 배운 것을 문화(2부), 프로세스(3부), 도구(4부)로 정리한 책이다. 원저자 세 명(Winters, Manshreck, Wright)은 구글의 C++ 라이브러리·도구·대규모 리팩터링을 실제로 한 사람들이다.

전체를 꿰는 논제는 1장에 있다: **소프트웨어 엔지니어링 = 프로그래밍을 시간 축으로 적분한 것**. 코드를 짜는 순간의 문제가 아니라, 코드가 살아있는 전 기간의 문제를 다루는 학문이라는 것. 그리고 이 논제를 지탱하는 세 개의 축이 책 전체에 반복된다:

표 1 책 전체를 관통하는 세 개의 축

축	질문	책에서의 예
시간 (time)	코드는 얼마나 오래 살 것인가, 변화에 어떻게 대응할 것인가	Hyrum의 법칙, deprecation, LSC, 테스트
규모 (scale)	조직·코드베이스가 커져도 효율이 유지되는가	모노레포, 빌드 시스템, 코드 검색, 셀프서비스
트레이드오프 (tradeoffs)	모든 결정에는 비용이 있다 — 근거를 가지고 선택했는가	생산성 측정, 스타일 가이드, 의존성 정책

읽는 법: 각 장은 ① 한 줄 논제 → ② 논증 전개 → ③ "기억할 것" 상자 순서로 구성했다. 원서 (abseil.io/resources/swe-book, 구글 공식 무료 공개)를 옆에 두고 병행하면 가장 좋다. 네 관심사(릴리즈·문서화·디자인)와 직결되는 장은 **10장(문서화)**, **15장(deprecation)**, **18장(빌드)**, **21장(의존성)**, **22장(LSC)**, **23장(CI)**, **24장(CD)** — 그래도 순서대로 읽기를 권한다. 이 책은 앞 장의 개념이 뒤 장의 전제가 되는 구조다.

제1부. 논제 (Thesis)

프로그래밍과 소프트웨어 엔지니어링의 차이는 코드가 아니라 시간이다.

1장. 소프트웨어 엔지니어링이란 무엇인가

한 줄 논제: "Software engineering is programming integrated over time." — 소프트웨어 엔지니어링은 시간에 걸쳐 적분된 프로그래밍이다.

프로그래밍은 코드를 생산하는 행위다. 소프트웨어 엔지니어링은 그 코드가 **예상 수명 동안, 기대보다 오래 살아남아, 수많은 사람이 건드리는 동안에도** 가치를 유지하게 하는 모든 활동을 더한 것이다. 대학 과제의 코드(수명: 1주)와 구글 검색의 코드(수명: 수십 년)는 같은 언어로 쓰이지만 필요한 것이 완전히 다르다. 이 차이를 설명하는 세 축이 이 장에서 나온다.

시간과 변화 — Hyrum의 법칙

코드의 예상 수명을 정하라는 책의 요구는 실용적이다: 1시간짜리 스크립트와 10년짜리 시스템은 다른 설계를 요구한다. 그리고 코드는 거의 항상 **예상보다 오래 산다**. 그래서 등장하는 것이 이 책에서 가장 유명한 법칙:

충분한 수의 사용자가 있는 API에서는, 네가 계약서에 무엇을 약속했든 상관없다. 시스템의 관찰 가능한 모든 동작은 누군가에게 의존성으로 잡힌다.

— Hyrum's Law (Hyrum Wright), 1장

문서화되지 않은 동작, 우연한 출력 순서, 암묵적인 타이밍 — 사용자가 그것에 의존하기 시작하는 순간 그것은 사실상 계약이 된다. 이 법칙 하나가 책의 후반부 전부를 예고한다: deprecation이 왜 어려운지(15장), SemVer가 왜 부족한지(21장), 리팩터링이 왜 도구 없이 불가능한지(22장).

규모와 효율 — 선의의 한계

엔지니어가 10명일 때는 "다들 잘하자"는 합의가 통한다. 10,000명이면 어떤 정책이든 매일 누군가는 처음 듣는다. 그래서 규모가 커지면 **정책은 문서가 아니라 자동화된 집행이어야 한다**는 것이 구글의 결론이다. 코드베이스 규모도 마찬가지: 한 사람이 전체를 이해할 수 없는 크기가 되면, 이해는 도구(코드 검색, 정적 분석, 테스트)가 보조해야 한다.

트레이드오프 — "왜"를 말할 수 있는 결정

구글은 "최선의 답"이 아니라 "근거 있는 선택"을 요구한다. 이 장의 미묘하지만 중요한 포인트: 결정의 질은 결과가 아니라 **결정 시점에 알려진 정보와 명시된 트레이드오프**로 평가한다. 그리고 문제를 조기에 발견할수록 수정 비용이 싸다(shift-left)는 원리가 나오는데, 이것이 테스트(11~14장)와 정적 분석(20장)의 경제적 근거다.

기억할 것

- ① 코드는 예상보다 오래 산다 — 수명을 먼저 정하고 설계하라.
- ② Hyrum의 법칙: 관찰 가능한 동작은 전부 사실상 계약이다.
- ③ 규모가 커지면 규칙은 사람의 합의가 아니라 자동화가 집행해야 한다.
- ④ 결정은 결과가 아니라 근거로 평가한다.

제2부. 문화 (Culture)

도구와 프로세스는 문화 위에서만 작동한다. 구글이 가장 먼저 투자하는 것.

2장. 팀에서 잘 일하는 법

한 줄 논제: 소프트웨어는 팀 스포츠다 — 혼자 빛나려는 습관이 팀 전체의 속도를 죽인다.

이 장의 표적은 **천재 신화(genius myth)**다. 우리는 유명한 소프트웨어를 한 명의 천재가 만든 것처럼 말하지만, 실제로는 거의 전부 팀의 산물이다. 이 신화가 해로운 이유: 주니어가 "나는 천재가 아니니까" 위축되고, 시니어가 "천재처럼 보여야 하니까" 미완성 작업을 숨기게 만든다.

숨기는 것(hiding)의 실제 비용: ① 혼자 오래 가면 설계 실수를 늦게 발견한다 — 되돌리는 비용이 기하급수로 커진다. ② **버스 팩터(bus factor)** — 지식이 한 사람 머리에만 있으면 그 사람이 빠지는 순간 프로젝트가 멈춘다. ③ 피드백 없는 고립은 실력 성장도 느리게 한다. 구글의 처방: **작게, 자주, 일찍 공유하라**. 미완성이란도 조기에 보여주는 것이 완성 후 보여주는 것보다 항상 낫다.

그리고 팀이 실패를 다루는 법: **비난 없는 포스트모템(blameless postmortem)**. 장애의 원인을 사람이 아니라 시스템에서 찾고, 배운 것을 문서로 남긴다. "왜 그 시스템은 그 실수를 하기 쉽게 설계됐는가"를 묻는 것이다. 이 장의 마지막 벽돌은 **심리적 안전감(psychological safety)** — 구글의 팀 효율성 연구에서 고성능 팀의 제1 요소로 꼽힌 것. 실수해도, 모른다고 해도, 반대해도 안전하다고 느끼는 팀만이 빠르게 배운다. 겸손(humility)·존중(respect)·신뢰(trust)이 그 기반이라고 이 장은 닫는다.

기억할 것

① 천재 신화는 팀을 망가뜨린다. ② 숨기지 마라 — 작게 자주 일찍 공유. ③ 버스 팩터를 재라. ④ 포스트모템은 사람이 아니라 시스템을 조사한다. ⑤ 심리적 안전감이 성능의 선행 조건.

3장. 지식 공유

한 줄 논제: 조직의 지식은 저절로 퍼지지 않는다 — 명시적으로 설계하고 투자해야 하는 인프라다.

2장의 버스 팩터를 조직 규모로 확장한 장이다. 지식이 특정인의 머리에만 있으면(부족 지식, tribal knowledge), 그 사람은 "단일 장애 지점"이다. 구글의 핵심 통찰: 지식 공유의 적은 기술이 아니라 **문화적 마찰**이다 — 사람들이 물기를 두려워하고, 문서를 쓰기를 귀찮아하고, 가르치는 시간을 낭비라고 여기는 순간 지식은 고인다.

그래서 먼저 심리적 안전감(2장)이 필요하다: **질문이 안전해야 지식이 흐른다**. 그 위에 구글은 채널을 용도별로 분리한다 — 문서(지속성 있는 정답), 채팅/메일링리스트(빠른 질의응답, 단 검색 가능해야), 코드 리뷰(맥락이 있는 1:1 지식 전달), office hours(전문가와의 정기 세션), 그리고 읽기 쉬운 코드 자체. 포인트는 **채널마다 수명이 다르다**는 것: 채팅에서 해결된 문제는 문서로 승격시켜야 다음 사람이 같은 질문을 안 한다.

조직 차원의 장치도 나온다: 신입 온보링 문서, Googler가 Googler를 가르치는 g2g 프로그램, 그리고 "가르치는 행위"를 승진 평가에서 인정하는 것. 마지막 논점이 중요하다 — 지식 공유는 개인의 미덕이 아니라 **보상 시스템이 뒷받침하는 조직 행동**이어야 한다. 문서 쓰고 남 가르치는 사람이 손해 보는 조직에서는 지식 공유가 절대 일어나지 않는다.

기억할 것

① 부족 지식 = 단일 장애 지점. ② 질문이 안전한 문화가 선행 조건. ③ 채널별 수명을 구분하고, 대화형 지식은 문서로 승격. ④ 지식 공유를 보상이 뒷받침하게 만들어라.

4장. 형평성을 위한 엔지니어링

한 줄 논제: 제품은 만든 사람의 편향을 물려받는다 — "모두를 위한" 소프트웨어는 의도적으로 설계해야 한다.

기술 챗터들 사이에서 특이한 위치의 장이지만, 논지는 엔지니어링적이다. 편향은 나쁜 의도 없이도 발생한다: 개발팀의 인구 구성이 사용자 인구 구성과 다르면, 팀이 "당연하다"고 여기는 가정이 특정 사용자를 배제한다. 이 장의 사례들(침착하게 서술됨): 특정 인종을 인식 못하는 이미지 인식, 여성 이름을 남성으로 처리하는 시스템, 접근성을 나중에 붙이다 실패한 제품들.

이 장의 핵심 주장 두 개. 첫째, **우리는 자기 편향을 스스로 발견하기 어렵다** — 그래서 다양한 배경의 사람이 결정 과정에 있어야 하고, 없다면 테스트와 데이터로 검증해야 한다. 둘째, 문제를 "개인의 의식"이 아니라 **환경과 프로세스**에서 고쳐라 — 채용 파이프라인, 성과 평가의 기준, 누가 리뷰어가 되는가 같은 시스템적 지점들. 이 책 전체의 방법론("사람 탓이 아니라 시스템을 고쳐라")이 인사·조직 영역에 적용된 장이라고 읽으면 일관된다.

기억할 것

① 편향은 악의 없이도 제품에 새어 나간다. ② 자기 편향은 스스로 못 본다 — 구성의 다양성과 검증이 필요. ③ 개인을 탓하지 말고 환경을 고쳐라.

5장. 팀을 이끄는 법

한 줄 논제: 리더십은 권한이 아니라 봉사다 — 리더의 일은 팀이 항해하기 쉽게 물살을 정리하는 것이다.

먼저 직책을 분해한다. **매니저**(사람의 성과·성장·복지를 책임), **테크 리드**(기술 방향과 결정을 책임), **TLM**(둘 다 — 스타트업/작은 조직에서 혼함). 구글의 경험칙: 두 역할의 긴장을 알고 분리할 수 있으면 분리하라 — 한 사람이 "기술적으로 옳은 것"과 "사람에게 좋은 것" 사이에서 매일 갈등하기 때문이다.

새 리더의 전형적 실패 목록이 이 장의 실전 가치다: ① 마이크로매니지먼트(불안해서 세부를 통제 — 팀의 자율성을 죽임), ② 저성과자 방치(팀 전체가 보고 있고, 고성과자부터 떠난다), ③ 끌리는 사람만 뽑기(편하지만 조직이 약해짐), ④ 사람 문제를 기술 문제로 오진. 이 장의 처방은 **서번트 리더십**: 리더의 작업은 명령이 아니라 장애물 제거, 방향 제시, 사람 성장 지원이다. "팀은 배다. 리더는 노를 젓는 사람이 아니라 방향을 잡고 물살을 정리하는 사람."

동기 부여에 관해서는 내재적 동기의 삼요소 — **자율성(autonomy)**, **숙달(mastery)**, **목적(purpose)** — 을 들고 온다. 외재적 보상(돈, 압박)은 단기에는 작동하지만 지식 노동의 창의성에는 역효과라는 것. 그리고 일대일 미팅, 피드백의 기술(구체적·행동 중심·타이밍 빠르게) 같은 기본기도 다룬다.

기억할 것

① 매니저와 테크 리드는 다른 직책이다. ② 마이크로매니지는 불안의 증상, 방치는 비용의 이전. ③ 리더는 장애물 제거자다. ④ 자율성·숙달·목적이 동기의 삼요소.

6장. 규모를 갖춰 이끌기

한 줄 논제: 조직이 커지면 리더의 작업도 스케일되어야 한다 — 결정·위임·자기 대체의 세 규칙으로.

이 장은 리더십 챕터 중 가장 독창적이다. 세 개의 만트라로 구성된다:

① **Always Be Deciding (항상 결정하라)**. 리더의 최악의 상태는 "결정 유예"다 — 팀은 멈추고, 선택지는 썩고, 하급자는 임의로 결정한다. 이 장의 결정 기술: 먼저 **질문이 맞는지** 검증하라(눈앞의 문제가 진짜 문제인가 — "이 문제를 다른 프레임으로 볼 수 있는가"). 트레이드오프를 명시하고, 완벽한 답이 없으면 반복 가능한 결정 절차로 가라. 결정 뒤에는 "결정의 근거"를 남겨라 — 미래의 리더가 같은 논쟁을 반복하지 않게.

② **Always Be Leaving (항상 떠날 준비를 하라)**. 역설적으로 들리지만 뜻은: 자신을 필요 없게 만들어라. 리더가 특정 문제의 유일한 해결사이면, 조직은 그 리더에게 단일 장애 지점을 갖게 된다(2장의 버스 팩터가 리더십 버전). 해결책은 위임 — "누가 이걸 대신 할 수 있나"를 항상 묻고, 후계자를 키우고, 자신은 다음 더 큰 문제로 이동하는 것. 조직도 마찬가지: 팀이 성숙하면 분할하고 각자의 리더를 세운다.

③ **Always Be Scaling (항상 스케일을 조정하라)**. 성공은 실패보다 다루기 어렵다 — 요청이 늘면 팀은 번아웃으로 죽는다. 이 장의 처방: 스스로 부하를 제한하라(무엇을 거절할지), 중요도와 긴급도를 분리하라(중요하지만 긴급하지 않은 일 — 문서화, 자동화 — 에 시간을 강제로 배정), 그리고 "이 문제의 실제 크기는 얼마인가"를 주기적으로 재평가하라.

기억할 것

① 결정 유예가 최악의 결정. 질문이 맞는지부터. ② 자신을 대체 가능하게 만드는 것이 리더의 일. ③ 성공의 스케일링이 실패보다 어렵다 — 거절이 보호다.

7장. 엔지니어링 생산성 측정

한 줄 논제: 생산성은 측정 가능하다 — 단, 개인이 아니라 팀·프로세스를, 직관이 아니라 프레임워크로.

구글은 엔지니어 생산성을 전담으로 연구하는 팀을 두고 있다. 이 장은 그 팀의 방법론이다. 먼저 전제: 측정의 목적은 개인 평가가 아니라 **"우리의 개선이 효과가 있었나"를 아는 것이다**. 그리고 생산성을 개인 지표(커밋 수, 코드 줄 수)로 재는 것은 명백히 실패한다 — 게임되고, 잘못된 행동을 보상한다.

방법론의 뼈대는 **GSM 프레임워크**: Goal(달성하고 싶은 목표를 문장으로) → Signal(목표가 달성됐을 때 관찰 가능해지는 변화) → Metric(시그널을 재는 지표). 예: 목표 "엔지니어가 코드 리뷰를 빨리 받는다" → 시그널 "리뷰 대기 시간이 줄어든다" → 지표 "리뷰 첫 응답까지의 중앙값 시간". 목표 없이 지표부터 만들면 쓰레기를 정밀하게 재게 된다.

측정 범주로는 **QUANTS**라는 다섯 분류를 제시한다: 코드의 품질(QUALity), 엔지니어의 주의(Ntention — 얼마나 방해받지 않고 몰입하는가), 지적 복잡성(복잡성 관리), 템포·속도(Tempo — 릴리즈·피드백 주기), 만족도(Satisfaction). 그리고 **삼각측량(triangulation)**: 정량 데이터(로그, 지표)와 정성 데이터(설문, 인터뷰)를 함께 봐야 한다 — 둘이 어긋나면 뭔가를 잘못 재고 있는 것이다.

기억할 것

① 측정은 팀/프로세스에, 개인 평가에 쓰지 마라. ② GSM: 목표→시그널→지표 순서를 지켜라. ③ 지표가 목표 자체가 되는 순간 그 지표는 죽는다. ④ 정량과 정성을 삼각측량하라.

제3부. 프로세스 (Processes)

문화가 "왜"라면 프로세스는 "어떻게, 매번, 누구나". 규모가 있는 조직의 일관성은 여기서 나온다.

8장. 스타일 가이드와 규칙

한 줄 논제: 스타일 가이드의 가치는 "이 규칙이 최선이라서"가 아니라 "모두가 같은 규칙을 따라서" 생긴다.

스타일 논쟁(탭 vs 스페이스 류)은 해결할 수 없다 — 하지만 **끝낼 수는 있다**. 구글의 스타일 가이드는 "이 방식이 우월하다"가 아니라 "이 방식으로 통일한다"는 선언이고, 그 통일 자체가 가치다: 코드베이스 어디를 열어도 같은 모양이면, 읽기 비용이 사라지고(17장의 코드 검색과 연결), 엔지니어가 팀을 옮겨도 관습을 새로 배울 필요가 없다.

규칙의 종류도 구분한다: ① 위험 방지 규칙(버그 패턴 금지 — 근거와 예외 조항이 명시됨), ② 일관성 규칙(임의의 선택이지만 통일), ③ 가이던스(권장이지만 강제 아님 — 판단의 여지). 중요한 운영 원칙: **규칙에는 근거가 문서화되어야 하고, 근거가 사라지면 규칙도 폐기된다**. 이것은 15장(deprecation)의 문화 버전이다.

그리고 1장의 "규모가 커지면 자동화가 집행한다"가 여기서 구체화된다: 포매퍼(clang-format, buildifier)가 스타일을 자동 적용하고, 정적 분석(20장)이 규칙 위반을 커밋 전에 잡는다. 사람 리뷰어는 스타일 지적에서 해방되어 설계 논의에 집중한다. **규칙의 집행이 자동일 때 비로소 규칙은 논쟁이 아니라 인프라가 된다**.

기억할 것

① 가치는 선택이 아니라 일관성에서 온다. ② 규칙에는 근거를 쓰고 근거가 죽으면 규칙을 폐기. ③ 집행은 사람이 아니라 포매퍼와 분석기가.

9장. 코드 리뷰

한 줄 논제: 코드 리뷰는 버그 잡기 이전에, 코드베이스의 이해를 조직 전체에 분산시키는 장치다.

리뷰의 가치를 구글은 셋으로 정리한다: ① 정확성(버그 조기 발견 — shift-left), ② 가독성("미래의 내가 읽을 수 있는가"를 검증 — 코드는 쓰이는 것보다 읽히는 횟수가 압도적으로 많다), ③ 일관성(8장의 규칙이 살아있는지 확인). 그리고 숨은 가치: **리뷰는 맥락 있는 지식 전달이다** — 주니어는 리뷰 코멘트로 배우고, 리뷰어는 남의 코드를 읽으며 코드베이스를 배운다(3장의 지식 공유와 연결).

프로세스의 뼈대: 모든 변경은 머지 전 리뷰를 거치고, 승인 권한은 **OWNERS** 파일로 디렉터리별로 선언된다(크로미움 실물에서 우리가 확인한 그것 — 구글 내부 시스템이 오픈소스에 그대로 이식됐다). 승인(LGTM)에는 두 가지가 필요하다: 가독성 승인(언어/스타일 관례에 맞는가)과 소유권 승인(그 코드의 오너가 동의하는가).

실전 원칙들: ① **변경은 작게** — 큰 CL은 리뷰 품질을 붕괴시킨다. 수백 줄이 넘으면 리뷰어는 "대충 LGTM"을 누른다. ② 리뷰 응답은 빠르게 — 리뷰 대기가 길면 개발자는 리뷰를 우회하기 시작한다(7장의 템포 측정이 여기 쓰인다). ③ 코멘트는 코드가 아니라 사람에게 — 질문형으로, 근거와 함께. ④ 리뷰에서 완벽주의 금지 — "개선"과 "요구"를 구분하라.

기억할 것

① 리뷰는 품질 장치이자 지식 분산 장치. ② 작은 변경이 리뷰를 살린다. ③ 리뷰 대기 시간은 프로세스의 건강 지표. ④ OWNERS = 승인 권한의 코드화.

10장. 문서화

한 줄 논제: 문서는 코드의 부산물이 아니라 코드의 일부다 — 같은 저장소, 같은 리뷰, 같은 생애주기로 다뤄라.

네가 이미 공부한 주제지만, 이 책의 버전은 구글 내부 시스템의 원조라 가치가 따로 있다. 이 장의 출발점: 엔지니어는 문서를 "코딩이 아닌 것"이라고 여겨 미루지만, 구글의 관점에서 **문서는 코드를 읽는 수천 명의 시간을 사는 투자**다 — 작성자 1시간이 독자 1,000시간을 절약한다.

핵심 메커니즘은 **docs-as-code**: 문서를 코드와 같은 버전 관리에 두고, PR로 리뷰하고, 코드 변경과 같은 변경 단위로 갱신한다. 문서가 별도 위키에 있으면 썩고, 코드 옆에 있으면 "코드 바꾸면서 문서 안 바꿨네?"가 리뷰에서 잡힌다(9장과 연결). 문서의 종류도 구분한다 — 튜토리얼, 레퍼런스, 설계 문서 — 그리고 각각 독자와 수명이 다르다(Diátaxis와 같은 사고방식이 구글 내부에서도 독립적으로 나타난 것).

주석에 대한 규칙은 크로미움 가이드라인과 정확히 일치한다: **주석은 "왜"를 설명하고, "무엇"은 코드가 말하게 하라.** 그리고 문서 품질의 지표는 양이 아니라 **신선도(freshness)** — 낡은 문서는 없는 것보다 나쁘다. 잘못된 지도는 지도가 없는 것보다 사람을 더 멀리 보내니까. 마지막으로: 문서 작성은 시니어의 의무다 — 주니어가 모르는 것을 아는 사람만이 쓸 수 있는 문서가 있다.

기억할 것

① 문서는 독자의 시간을 사는 투자. ② docs-as-code: 같은 저장소·리뷰·생애주기. ③ 주석은 "왜"만. ④ 지표는 양이 아니라 신선도.

11장. 테스트 개관

한 줄 논제: 테스트는 버그를 찾는 활동이 아니라, 변경을 가능하게 하는 인프라다.

이 장의 재정의: 테스트의 진짜 가치는 버그 발견이 아니라 **개발 속도**다. 테스트가 없는 코드베이스는 시간이 갈수록 변경이 느려진다 — 무엇을 깨뜨릴지 몰라서. 테스트가 있는 코드베이스는 리팩터링과 업그레이드가 일상이 된다(21장 Live at Head와 22장 LSC의 전제 조건이 바로 이것). 구글이 모든 변경에 테스트를 요구하는 이유: "테스트 없는 코드는 레거시 코드다."

테스트의 분류는 **크기(size)**로 한다 — 작은 테스트(단일 프로세스, 빠름, 결정적), 중간 테스트(단일 머신), 큰 테스트(여러 머신, 실제 환경에 가까움). 그리고 비율의 지침이 유명한 **테스트 피라미드**: 단위 테스트 80%, 통합 15%, 엔드투엔드 5%. 작을수록 빠르고 결정적이고 실패 원인이 명확하고, 클수록 실제 동작에 가깝지만 느리고 불안정하다. 피라미드가 뒤집히면(아이스크림 콘) CI가 느려지고 flaky가 폭증한다.

실무 원칙들: ① 테스트는 **헤록매틱**해야 한다 — 외부 시계, 네트워크, 실행 순서에 의존하는 테스트는 flaky의 온상. ② 커버리지는 지표지 목표가 아니다 — 100% 커버리지가 보증하는 건 "실행됐다"뿐이지 "검증했다"가 아니다. ③ **비윤세 규칙**: "좋았으면 테스트를 붙였어야지(If you liked it, you shoulda put a test on it)" — 인프라 팀이 변경했을 때 네 코드가 깨졌다면, 테스트가 없던 네 책임이다. ④ 테스트는 실행 가능한 문서다 — 잘 쓰인 테스트가 API 사용법의 가장 정확한 명세다.

기억할 것

① 테스트는 속도의 인프라다. ② 피라미드 80/15/5. ③ 헤르메틱 테스트만 신뢰할 수 있다. ④ 비윤세 규칙: 테스트 없는 의존은 묵시적 동의.

12장. 단위 테스트

한 줄 논제: 좋은 단위 테스트의 기준은 커버리지가 아니라 유지보수 가능성이다 — 테스트도 코드다.

단위 테스트가 가장 흔하게 망하는 경로는 **깨지기 쉬운(brittle) 테스트**다: 리팩터링 때마다 테스트가 대거 실패하고, 팀은 "테스트가 짐"이라는 결론에 도달한다. 이 장의 진단: 깨지기 쉬운의 원인은 거의 항상 **구현 디테일을 테스트했기 때문이다**. 테스트는 공개 API의 동작을 검증해야 한다 — 내부 메서드, 호출 순서, 내부 상태를 검증하면, 동작이 그대로인 리팩터링에도 테스트가 깨진다. "테스트는 동작이 바뀔 때만 깨져야 한다."

테스트의 명확성 원칙이 이 장의 실전 핵심: **DAMP over DRY**. 프로덕션 코드에서는 중복 제거(DRY)가 미덕이지만, 테스트에서는 서술적이고 의미 있는 문구(DAMP: Descriptive And Meaningful Phrases)가 우선이다. 테스트는 실패했을 때 **테스트 코드만 읽고 원인을 알아야** 한다 — 공유 setup에 초기화가 흩어져 있으면 실패한 테스트 하나를 이해하려고 파일 전체를 훑어야 한다. 그래서 구글은 테스트마다 필요한 데이터를 눈에 보이게 적는 것(약간의 중복을 감수)을 권장한다.

구조의 표준: **given/when/then** — 준비(상태 구성), 실행(대상 호출), 검증(결과 단언)의 삼단 구조. 그리고 하나의 테스트는 하나의 동작만 검증한다: 테스트 이름이 "A_should_B_when_C" 형태가 되면, 실패 리포트만 봐도 무엇이 깨졌는지 알 수 있다. 헬퍼 메서드는 중복이 아니라 **관련 없는 디테일을 숨기는 용도**로만 쓴다.

기억할 것

① 공개 API의 동작만 테스트하라 — 구현 디테일은 금지. ② 테스트는 DAMP하게: 실패 시 그 자리에서 읽혀야 한다. ③ given/when/then, 한 테스트 한 동작. ④ 깨지기 쉬운 테스트는 테스트가 아니라 부채다.

13장. 테스트 더블

한 줄 논제: 테스트 더블(가짜 의존성)은 필요악이다 — 진짜를 쓸 수 있으면 진짜를 써라.

용어 정리부터: **fake**(실제와 비슷하게 동작하는 경량 구현 — 인메모리 DB), **stub**(정해진 값만 반환), **mock**(어떤 호출이 일어났는지 검증하는 객체). 이 장의 계급이 명확하다: **실제 구현 > fake > stub > mocking 프레임워크**.

왜 실제 구현이 우선인가: 가짜는 진짜와 다르다 — 다르다는 것은, 테스트가 통과해도 프로덕션에서 실패할 구멍이 있다는 뜻이다(테스트의 충실도, fidelity 문제). 그럼 왜 더블을 쓰나: 실재가 너무 느리거나(원격 API), 비결정적이거나(시간, 랜덤), 위험할 때(결제 시스템). 이때의 선호는 fake — 실제 계약을 충실히 구현한 경량 버전이기 때문이다. fake는 한 번 만들면 모든 테스트가 재사용한다.

mocking 프레임워크에 대한 경고가 이 장의 정점이다: mock으로 상호작용을 검증하면("이 메서드가 이 인자로 정확히 한 번 호출됐는가") 테스트는 구현 디테일에 용접된다 — 12장의 brittle 문제를 극단으로 만든다. 이것 **과잉 명세(over-specified) 테스트**라 한다. 구글의 교훈: 상태 검증(결과가 무엇인가)이 상

호작용 검증(누구를 불렀는가)보다 거의 항상 낮고, 상호작용 검증이 정말 필요한 경우(부작용 자체가 계약인 경우 — 예: 이메일 발송)로 한정하라.

기억할 것

① 선호 순서: 실제 > fake > stub > mock. ② 가짜와 진짜의 차이 = 테스트가 못 보는 구멍. ③ 상태 검증 > 상호작용 검증. ④ mock 남용 = 구현에 용접된 테스트.

14장. 더 큰 테스트

한 줄 논제: 큰 테스트(통합·E2E)는 피라미드 꼭짓점의 양념이지 주식이 아니다 — 필요한 곳에 정확히, 불안정성은 인프라로 관리한다.

단위 테스트가 아무리 좋아도 증명 못 하는 것이 있다: **부품들이 함께 동작하는가**. 큰 테스트는 충실도(실제 환경과의 유사도)를 올리는 대신 세 가지 비용을 지불한다: 느림(초·분 단위), 비결정성(flaky — 타이밍, 리소스, 순서 의존), 디버깅의 어려움(실패 원인의 반경이 넓음). 이 비용 구조 때문에 피라미드 상단은 좁아야 한다(11장).

구글이 큰 테스트의 불안정성과 싸우는 방식이 인프라적이다: ① **헤록매틱 테스트 환경** — 테스트 전용 서버를 띄워 외부 세계를 격리. ② **flaky의 자동 관리** — 불안정한 테스트를 자동 격리(quarantine)하고 소유자에게 할당. flaky를 "재시도하면 통과하니까"로 방치하면 테스트 스위트 전체의 신뢰가 죽는다 — 아무도 빨간색을 믿지 않게 되는 것이 최악의 상태. ③ 큰 테스트도 결국 코드다 — 유지비를 의식하고, 시나리오 수가 아니라 **위험이 큰 경로**를 고른다.

언제 큰 테스트를 쓰는가: 컴포넌트 경계의 계약 검증, 복원 불가능한 동작(데이터 마이그레이션), 그리고 단위 테스트로 표현 불가능한 상호작용. 피라미드 상단의 후보는 "여기가 깨지면 큰일나는" 경로여야 하지, "테스트하기 쉬운" 경로가 아니다.

기억할 것

① 큰 테스트는 충실도를 사고 속도·안정성을 판다. ② flaky는 재시도가 아니라 격리와 원인 제거로. ③ 신뢰 못하는 테스트 스위트는 없는 것보다 나쁘다. ④ 위험한 경로를 고르고, 쉬운 경로를 고르지 마라.

15장. 폐기(Deprecation)

한 줄 논제: 시스템을 진화시키는 것은 새 것을 만드는 능력이 아니라, 낡은 것을 제거하는 능력이다.

이 책에서 가장 과소평가된 장. 소프트웨어 조직은 새 API 만드는 것은 잘하는데, **낡은 API의 완전한 제거**에는 반복적으로 실패한다 — 그 결과 시스템은 층층이 쌓인 레거시의 고고학 유적이 된다. deprecation이 어려운 이유는 1장으로 돌아간다: **Hyrum의 법칙**. 문서화한 계약이 아니라 관찰 가능한 모든 동작에 누군가 의존하므로, "아무도 안 쓰는 것"을 증명하는 것조차 어렵다.

구글의 deprecation 절차: ① **측정** — 낡은 것의 실제 사용량을 코드 검색과 로그로 파악. ② **경고** — 사용 시점에 폐기 예고를 노출(컴파일 경고, 런타임 로그, 문서). ③ **마이그레이션 지원** — 새로운 것으로의 이주를 기계적으로 돕는다(자동 변환 도구, 22장의 LSC와 직결). ④ **삭제** — 잔존 사용량이 0이 될 때까지 추적하고, 실제로 지운다. 이 ④까지 완료해야 deprecation이 끝난 것이다 — "폐기 예정" 표시만 하고 방치하는 것은 deprecation이 아니라 장식이다.

조직적 포인트: 마이그레이션 비용을 **낮은 것을 만든 팀(인프라 팀)이 지는 것이** 구글 모델이다. 사용자 팀에게 "알아서 옮겨라"라고 하면 아무도 안 옮긴다. 그리고 낮은 것과 새 것의 공존 기간 동안, 낮은 것에 신규 사용이 생기지 않게 막는 장치(lint 규칙 — "이 클래스 신규 import 금지")가 필요하다. 흡수저 우물 파는 동안 물이 다시 차는 것을 막는 것이다.

기억할 것

① 제거까지가 deprecation이다. ② 측정→경고→마이그레이션 지원→삭제. ③ 이주 비용은 인프라 팀이 진다. ④ 신규 사용 차단 없는 폐기는 영원히 안 끝난다.

제4부. 도구 (Tools)

문화와 프로세스를 기계로 굳히는 층. 구글의 도구들은 "좋아서" 쓰는 게 아니라 "규모가 강제해서" 생겼다.

16장. 버전 관리와 브랜치 관리

한 줄 논제: 진실의 원천(source of truth)은 하나여야 한다 — 그리고 구글은 그것을 저장소 하나(monorepo)와 트렁크 하나로 구현한다.

버전 관리의 본질적 질문: "정답인 코드가 어디 있는가"를 조직 전체가 알 수 있는가. 이 장은 구글의 답인 **모노레포**를 변호한다. 모노레포의 논거: ① 투명성 — 모든 코드가 모든 엔지니어에게 보인다(검색 가능, 17장). ② 하나의 최신 — "어떤 버전 조합이 맞는가"라는 의존성 지옥이 내부 코드에는 존재하지 않는다(21장). ③ 원자적 변경과 대규모 변경의 가능성 — 수천 파일을 한 번에 고치는 LSC(22장)는 모노레포로서 가능하다.

브랜치 정책: **트렁크 기반 개발**. 개발 브랜치는 짧게(며칠), 머지는 빠르게. 장기 개발 브랜치의 문제는 수학이다 — 브랜치가 오래 살수록 머지 충돌은 확률이 아니라 확정이 되고, 충돌 해결 비용은 지수적으로 자란다. 대신 미완성 기능은 브랜치가 아니라 **피쳐 플래그**로 숨긴다(배포 ≠ 릴리즈 분리). 브랜치는 두 가지 용도로만: 릴리즈 브랜치(체리픽으로 안정화)와 핫픽스.

모노레포의 한계도 정직하게 다룬다: 수십억 줄을 일반 VCS는 못 받는다(구글이 Piper라는 자체 시스템을 만든 이유), 그리고 모노레포는 도구 투자가 선행돼야 한다. 교훈은 "모노레포를 하라"가 아니라 **"진실의 원천을 하나로 유지하라 — 그리고 그 결정의 비용을 도구로 지불하라"**다.

기억할 것

① source of truth는 하나. ② 트렁크 기반: 브랜치는 짧게, 미완성은 플래그로. ③ 모노레포의 진짜 가치는 투명성과 원자적 변경. ④ 모노레포는 결정이 아니라 도구 투자 패키지다.

17장. 코드 검색

한 줄 논제: 코드베이스가 한 사람의 기억을 넘어서면, 검색이 이해의 유일한 통로다 — 코드 검색은 사치가 아니라 핵심 인프라다.

구글 엔지니어가 코드를 "읽는" 주된 방식은 IDE가 아니라 Code Search라는 전용 도구다(오픈소스 후계가 Kythe와 크로미움의 codesearch). 왜 필요한가: 모노레포 전체를 로컬에 받을 수 없고, 받아도 grep은 느리고, 의미를 모른다(이 심볼의 정의가 어디고 참조가 어디인지). 코드 검색은 전체 코드베이스를 인덱싱해서, 심볼의 정의·참조·이력을 초 단위로 답한다.

용도의 계층: ① **이해** — "이 함수가 뭘 하는가"를 실제 코드로 확인(문서는 썩어도 코드는 진실). ② **영향 분석** — "이 API를 누가 쓰는가"를 전수 조사. 15장 deprecation의 사용량 측정과 22장 LSC의 대상 선정이 전부 이 도구 위에 있다. ③ **소유자 찾기** — OWNERS와 git 이력으로 "누구에게 물어봐야 하는가"를 해결.

전략적 교훈: 코드 이해의 비용은 작성 비용을 수십 배 초과한다 — 코드는 수십 번 쓰이지만 수천 번 읽힌다. 그래서 "읽기 비용을 낮추는 인프라"(검색, 네비게이션, 크로스 레퍼런스)에 대한 투자는 전사 생산성 투자다. 3장의 지식 공유가 도구로 구현된 사례로 읽어라.

기억할 것

- ① 기억력의 한계 이후, 이해는 검색이 담당한다. ② 사용처 전수 조사가 deprecation과 LSC의 기반.
- ③ 읽기 비용 ↓ = 조직 속도 ↑.

18장. 빌드 시스템과 빌드 철학

한 줄 논제: 빌드 시스템은 스크립트가 아니라 의존성의 공학이다 — 규모가 커지면 빌드의 속도와 정확성이 조직의 속도를 결정한다.

빌드 도구의 세대를 구분한다. **태스크 기반**(make, Ant): "이 순서로 이 명령을 실행해" — 유연하지만 의존성을 도구가 모른다. 명시 안 된 의존성(헤더 파일, 생성 코드)에 결과가 오염되고, 증분 빌드가 틀리고, 결국 `make clean`을 신봉하게 된다. **아티팩트 기반**(Bazel, Buck): 빌드 대상을 의존 그래프로 선언하면 도구가 필요한 것만 정확히 다시 계산한다.

Bazel(내부 이름 Blaze)의 설계 원칙: ① **선언적 BUILD 파일** — 무엇을 빌드하는가만 선언하고, 방법은 도구가 결정. ② **헤르메틱(밀폐) 실행** — 모든 입력을 명시하고 샌드박스에서 실행. 같은 입력은 같은 출력. 이것이 16장의 재현성과 캐시의 전제. ③ **리모트 캐시·실행** — 결과를 콘텐츠 해시로 전사 공유하고, 워커 클러스터에서 병렬 실행. "동료가 이미 빌드한 결과를 다시 계산하지 않는다."

규모의 함정도 다룬다: 의존 그래프 자체가 관리 대상이다 — 지나치게 큰 모듈(재사용 불가, 전부 다시 빌드)과 지나치게 잘게 쪼갠 모듈(그래프 관리 비용) 사이의 균형. 그리고 외부 의존성: 내부 모노레포는 "one-version rule"(같은 라이브러리는 하나의 버전만)을 지키고, 이것이 21장의 Live at Head로 이어진다. 빌드 철학의 한 줄: **빌드가 빠르고 정확하면 엔지니어는 작게 자주 변경한다 — 빌드가 느리고 불확실하면 변경이 커지고 위험해진다.** 빌드 시스템은 개발자 행동의 설계다.

기억할 것

- ① 태스크 기반의 한계 = 도구가 의존성을 모름. ② 헤르메틱 = 재현성 = 캐시의 전제. ③ 의존 그래프는 자산이자 관리 대상. ④ 빌드 성능이 개발자 행동을 설계한다.

19장. Critique: 구글의 코드 리뷰 도구

한 줄 논제: 리뷰 도구는 diff 뷰어가 아니라, 리뷰라는 사회적 프로세스의 마찰을 제거하는 장치다.

9장의 프로세스가 도구로 구현된 사례. Critique(구글 내부 리뷰 도구)의 설계가 말하는 것: 리뷰의 병목은 기술이 아니라 **대기와 오해**다. 그래서 도구가 하는 일: ① 리뷰어 추천(OWNERS + 이력 기반 — "누구에게 보내야 하는가"의 자동화), ② 정적 분석 결과를 리뷰 화면에 통합(20장의 Tricorder — 사람이 스타일 지적을 안 해도 됨), ③ 코멘트의 상태 관리(해결됨/미해결 — 사회적 합의의 가시화), ④ LGTM과 승인 권한의 명시적 표시.

이 장을 읽는 시니어의 눈: 도구 선정의 기준은 기능 목록이 아니라 **"우리 프로세스의 어떤 마찰을 없애는가"**다. 리뷰가 느린 조직은 도구 전에 프로세스를 봐야 하고(CL 크기, OWNERS 명확성), 프로세스가 건강한 조직은 도구가 남은 마찰을 없앤다. Gerrit, Reviewable, GitHub PR도 같은 문제를 각자의 답으로 푼다.

기억할 것

① 리뷰의 병목은 대기와 오해 — 도구는 마찰 제거기. ② 분석 결과는 기계가, 판단은 사람이. ③ 도구는 프로세스의 반영이지 대체가 아니다.

20장. 정적 분석

한 줄 논제: 컴파일러가 잡는 것보다 한 단계 위의 버그를, 사람 리뷰 전에 기계가 잡는다 — 단, 신호대잡음비가 생명이다.

정적 분석의 가치는 shift-left의 극치다: 테스트도 실행 전, 심지어 컴파일 시점에 버그를 잡는다. 하지만 이 장의 진짜 교훈은 기술이 아니라 **채택률의 공학**이다. 정적 분석 도구의 실패 패턴은 하나로 수렴한다: **오탐(false positive)**. 경고의 10%가 쓸모없으면 개발자는 100%를 무시하기 시작한다. 그래서 구글의 원칙: 경고는 "지금 고쳐야 할 것"만 — 확신 없는 것은 경고가 아니라 정보로 격을 낮추고, 오탐률이 높은 검사는 도입하지 않는다.

인프라 구조: ① 컴파일러가 제1 분석기(경고를 예외로 승격할 수 있는 것만 경고로). ② **Error Prone**(Java), Clang-Tidy — 컴파일 파이프라인에 흑인 패턴 검사, 자동 수정 제안까지. ③ **Tricorder** — 분석 결과를 코드 리뷰 화면에 표시하는 플랫폼. 분석기를 누구든 기여할 수 있고, 결과는 리뷰 컨텍스트에서 본다. 포인트: **분석은 개발자의 작업 흐름 안에 있어야 한다** — 별도 대시보드에 있는 경고는 존재하지 않는 것과 같다.

자동 수정의 경제학: 정적 분석이 발견→수정 제안→적용까지 가면, 22장의 LSC가 가능해진다. "이 패턴을 전사 코드에서 고쳐라"를 사람이 아니라 도구가 하는 것. 정적 분석은 버그 탐지 도구가 아니라 **코드베이스를 진화시키는 기계**의 입력 장치다.

기억할 것

① 오탐 10%면 전체가 무시된다 — 신호대잡음비가 전부. ② 경고는 작업 흐름 안에, 별도 대시보드 금지. ③ 발견→자동 수정→LSC로 이어지는 것이 최종형.

21장. 의존성 관리

한 줄 논제: 의존성 관리의 진짜 문제는 버전 고르기가 아니라, "여러 버전이 공존할 수 없다"는 제약을 조직이 어떻게 처리할 것인가다.

의존성 지옥의 핵심 메커니즘: **다이아몬드 의존성**. A가 B와 C에 의존하고, B와 C가 각각 다른 버전의 D를 요구하면 — 대부분의 생태계에서 D는 하나만 로드될 수 있어서, 둘 중 하나는 요구를 못 이룬다. 이 문제의 크기는 라이브러리 수의 선형이 아니라 조합의 폭발로 자란다.

이 장은 네 가지 전략을 비교한다. ① **아무것도 안 함**(항상 최신) — 처음엔 편하고 나중엔 불안정. ② **SemVer** — 호환성 약속에 기대는 타협. 유용하지만 두 가지 한계: 약속이 지켜진다는 보장이 없고, Hyrum의 법칙에 의해 "문서화된 계약"보다 실제 의존이 항상 넓다. ③ **번들링**(각자 의존성을 내장) — 충돌은 피하지만 보안 패치가 전파되지 않는 재앙. ④ **Live at Head** — 모든 의존성을 최신 커밋에 맞춘 상태로 유지. 구글의 답이다.

Live at Head의 논리: 버전 고정 = 비용을 미루는 것이지 없애는 것이 아니다 — 업그레이드를 미룰수록 차이가 쌓여 이주 비용이 지수로 자란다. 차라리 **업그레이드를 지속적으로 조금씩** 하면 비용이 상수가 된

다. 단, 이 모델은 세 가지 전제 없이는 불가능하다: 모노레포(16장), 전수 테스트(11~14장), 그리고 마이그레이션을 대신 수행하는 기계(22장 LSC). "의존성 관리"가 사실은 테스트·빌드·변경 도구의 종합 시험인 이유다. 오픈소스 세계와의 비교도 나온다: Go 모듈의 최소 버전 선택(MVS)처럼, 생태계는 SemVer의 타협 위에서 각자의 답을 찾는다.

기억할 것

- ① 다이아몬드 의존성 = 하나만 로드되는 제약. ② SemVer는 타협이지 해결책이 아니다. ③ 미룬 업그레이드는 지수 비용. ④ Live at Head는 정책이 아니라 인프라 패키지다.

22장. 대규모 변경(LSC)

한 줄 논제: 수천 파일에 걸친 변경을 "원자적으로 못 하는" 현실에서, 분할·자동화·사회적 합의로 수행하는 기술.

LSC(Large-Scale Change)의 정의: 논리적으로는 하나지만 물리적으로는 하나의 커밋으로 불가능한 변경 — 예: "전사 코드에서 deprecated API를 새 API로 교체", "로깅 라이브러리 전환". 왜 어려운가: ① 한 번에 커밋할 수 없는 크기(빌드·테스트가 깨짐), ② 한 사람이 리뷰할 수 없는 범위, ③ 수백 팀의 우선순위와 협상해야 하는 사회적 문제.

구글의 핵심 조직 원칙이 이 장의 정수다: **마이그레이션은 사용자 팀이 아니라 인프라 팀이 수행한다.** "모든 팀이 각자 고쳐라"는 방식은 반드시 실패한다(15장 — 아무도 안 옮김). 그래서 중앙 팀이 도구로 변경을 생성하고, 소유자는 리뷰만 한다. 이걸 가능케 하는 기계가 **Rosie**: 변경을 디렉터리별로 분할하고, 각 조각에 테스트를 돌리고, OWNERS에게 자동으로 리뷰를 보내고, 승인되면 머지하는 파이프라인. 사람이 하는 것: 변경 스크립트 작성과 최종 승인뿐이다.

LSC의 절차적 교훈: ① 작게 나눠라(조각마다 독립적으로 안전하게), ② 테스트가 전제 조건(테스트 없는 코드는 LSC도 못 건드릴 — 비윤세 규칙이 여기서 회수된다), ③ 마이그레이션 기간 동안 신규 사용을 막아라(15장의 금지 규칙), ④ 완료까지 추적하라. 그리고 철학적 한 줄: **구글에서 인프라 개선은 "할 수 있다"가 아니라 "끝까지 간다"다** — LSC 능력이 있어야 deprecation이 완결되고, deprecation이 완결돼야 코드베이스가 썩지 않는다.

기억할 것

- ① LSC = 논리적 하나, 물리적 수백 개. ② 마이그레이션은 인프라 팀의 일. ③ 테스트가 LSC의 입장권. ④ 나누고, 자동화하고, 끝까지 추적하라.

23장. 지속적 통합(CI)

한 줄 논제: CI는 "자동 빌드"가 아니라, 코드베이스가 항상 출시 가능 상태에 있다는 것을 지속적으로 증명하는 시스템이다.

CI의 경제적 근거는 1장의 shift-left다: 문제는 트렁크에서 멀어질수록 비싸진다. 머지 전에 잡으면 몇 분, 머지 후에 잡으면 "누구의 변경인가" 수사가 필요하고, 릴리즈 후에 잡으면 사용자 피해. 그래서 검증을 시간축으로 배치한다: **presubmit**(머지 전 — 빠르고 확실한 것만, 수 분 내 피드백)과 **postsubmit**(머지 후 — 느리고 넓은 것, 전체 통합 테스트). 모든 테스트를 presubmit에 넣고 싶은 유혹을 이겨라 — presubmit이 느려지면 개발자가 우회한다.

규모에서의 문제: 구글 크기가 되면 "모든 테스트를 매 커밋에"는 계산량이 불가능하다. 그래서 **영향 분석 기반 선별** — 빌드 그래프(18장)에서 "이 변경이 영향 주는 타겟"만 테스트한다(TAP이라는 시스템). 이것이 해록매틱 빌드와 정확한 의존 그래프가 선행돼야 하는 이유의 실전 사례다: 그래프가 정확해야 "영향 없음"을 믿고 테스트를 생략할 수 있다.

CI의 신뢰 관리: 빨간 상태(테스트 실패)는 즉시 최우선으로 처리한다 — "빨간 게 정상"이 되는 순간 CI는 장식이 된다. flaky 테스트는 격리하고(14장), 실패 해석 비용을 낮추는 것이 CI 팀의 일이다. 그리고 릴리즈 후보 관리: CI가 그린 커밋 중에서 릴리즈 후보를 고르고, 핫픽스는 릴리즈 브랜치에 체리픽(5장의 트레인과 연결). CI의 산출물은 "통과"가 아니라 "언제든 출시할 수 있는 트렁크"다.

기억할 것

① presubmit은 빠르게, postsubmit은 넓게. ② 규모에서는 영향 분석으로 선별 테스트. ③ 빨간 상태의 방치 = CI의 죽음. ④ CI의 산출물은 출시 가능한 트렁크.

24장. 지속적 배포(CD)

한 줄 논제: 배포 속도는 팀의 근면함이 아니라 파이프라인의 품질로 결정된다 — 그리고 배포와 릴리즈를 분리하는 순간 속도와 안정이 공존한다.

이 장의 출발점: 왜 배포가 느린가를 볼 때 사람을 보지 말고 파이프라인을 보라. 수동 단계, 불명확한 책임, "배포 요일" 같은 인공 장벽이 병목이다. CD의 전제 조건들: ① 신뢰 가능한 CI(23장 — 이게 없으면 CD는 자동 장애 배포), ② **버전된 불변 아티팩트** — 빌드는 한 번, 그 결과물이 환경을 이동하며 승격(환경마다 다시 빌드하지 않는다), ③ **역등 배포** — 같은 배포를 두 번 해도 안전, ④ 롤백이 일상 절차.

그리고 이 책의 CD 철학을 관통하는 분리: **배포(코드를 프로덕션에 두는 것)와 릴리즈(사용자에게 보이는 것)는 다른 결정이다**. 피치 플레그로 이 둘을 분리하면, 배포는 지속적으로(위험 ↓ — 변경이 작으므로), 릴리즈는 점진적으로(칸나리 → 확대, 관찰하며) 할 수 있다. "지속적 배포"와 "지속적 전달(continuous delivery)"의 용어 구분도 여기서: delivery는 출시 가능 상태 유지, deployment는 실제 자동 출시.

조직적 교훈: **릴리즈 프로세스를 제품처럼 다뤄라**. 배포 경험에도 사용자(개발자)가 있고, 지표(DORA 4 keys)가 있고, 개선 로드맵이 있다. 그리고 반직관의 결론(후에 DORA 연구로 실증됨): 속도와 안정은 트레이드오프가 아니다 — 작은 변경·자동 검증·빠른 롤백을 갖춘 조직이 둘 다 이긴다.

기억할 것

① 배포 속도는 파이프라인 품질의 함수. ② 빌드는 한 번, 아티팩트가 승격한다. ③ 배포 ≠ 릴리즈: 플레그로 분리. ④ 릴리즈 프로세스를 제품으로 다뤄라.

25장. 서비스로서의 컴퓨트

한 줄 논제: 머신을 소유하는 사고에서, 컴퓨팅을 유틸리티로 소비하는 사고로 — 이 전환이 조직의 소유권 경계를 다시 그린다.

마지막 장은 인프라의 추상화 계층을 다룬다. 구글의 컴퓨트 스택의 진화: 물리 머신 → 가상 머신 → 컨테이너(Borg — 쿠버네티스의 내부 원조) → 서버리스. 각 계층이 하는 일은 같다: **아래 계층의 관리를 위임하고 위의 관심사에 집중한다**. Borg의 통찰: 수만 대의 머신을 "누가 어디서 실행할지" 사람이 배정하는 것

이 아니라, 스케줄러가 선언적 요구("이 작업이 CPU 2개 필요")를 보고 자동 배치(bin packing) — 데이터센터를 하나의 컴퓨터처럼 다룬다.

엔지니어링 조직에의 함의가 이 장을 읽는 이유다: 컴퓨트가 서비스가 되면, "내 서버"라는 소유 개념이 사라지고 **운영 책임의 경계**가 다시 그어진다 — 플랫폼 팀은 머신을, 제품 팀은 코드를 소유한다. 이 분리가 24장의 셀프서비스 배포와 만나 완성된다: 제품 팀이 머신을 몰라도 배포할 수 있을 때, 조직 전체의 속도가 나온다. 주의할 트레이드오프: 추상화는 편의와 비용의 교환이고, "누가 새벽 3시에 깨는가"를 계약으로 명확히 해야 한다.

기억할 것

① 컴퓨트는 소유물에서 유틸리티로. ② 선언적 요구 + 스케줄러 = 데이터센터가 하나의 컴퓨터. ③ 추상화 계층 = 소유권 경계의 재설계.

나가며: 이 책의 한 줄

25장을 관통하는 문장을 하나로 압축하면 이것이다:

소프트웨어 엔지니어링은, 시간과 규모가 만드는 문제를, 문화·프로세스·도구의 삼각대로 다루는 것이다.

문화가 없으면 프로세스는 형식이 되고, 프로세스가 없으면 도구는 방치되고, 도구가 없으면 문화는 선언에 그친다. 이 책의 모든 장은 셋 중 하나의 이야기이면서 동시에 다른 둘을 전제로 한다 — 테스트(문화: 비윤세 규칙 / 프로세스: presubmit / 도구: TAP)가 그 삼각 구조의 표본이다.

실무자에게 남는 적용 순서: ① 먼저 측정 가능한 병목 하나를 고르고(7장 GSM), ② 그것을 프로세스 규칙으로 만들고(8~9장), ③ 규칙을 기계가 집행하게 하라(18~20장). 사람에게 "잘하자"고 말하는 순간 이 책의 첫 페이지로 돌아가라.

부록. 25장 색인표

표 2 장별 핵심 개념 색인

장	한 줄 핵심
1장	소프트웨어 엔지니어링 = 시간에 적분된 프로그래밍. Hyrum의 법칙.
2장	천재 신화 부정, 숨기지 않기, 버스 팩터, 심리적 안전감.
3장	지식 공유는 인프라. 채널 분리, 문서로의 승격, 보상 연계.
4장	편향은 시스템 문제. 환경을 고치고 다양성으로 검증.
5장	매니저 ≠ 테크리드. 서번트 리더십. 자율성·숙달·목적.
6장	항상 결정하고, 항상 떠날 준비하고, 항상 스케일 조정하라.
7장	GSM(목표→시그널→지표), QUANTS, 삼각측량. 개인 측정 금지.
8장	가치는 일관성. 규칙에는 근거, 집행은 자동화.
9장	리뷰 = 품질 + 지식 분산. 작은 CL, 빠른 응답, OWNERS.
10장	문서는 코드의 일부. docs-as-code, 주석은 "왜", 신선도.
11장	테스트는 속도의 인프라. 피라미드 80/15/5, 헤ureka매틱, 비윤세 규칙.
12장	공개 API만 테스트. DAMP over DRY. given/when/then.
13장	실제 > fake > stub > mock. 상태 검증 우선.
14장	큰 테스트는 위험 경로에만. flaky는 격리와 원인 제거.
15장	측정→경고→마이그레이션→삭제. 제거까지가 deprecation.
16장	진실의 원천은 하나. 모노레포 + 트렁크 기반 + 플러그.

17장	코드 검색 = 이해·영향 분석·소유자 찾기의 인프라.
18장	아티팩트 기반 빌드, 헤르메틱, 리모트 캐시. 빌드가 행동을 설계.
19장	리뷰 도구 = 마찰 제거. 분석 통합, 승인의 가시화.
20장	정적 분석은 신호대잡음비가 생명. 작업 흐름 안에 배치.
21장	다이아몬드 의존성, SemVer의 한계, Live at Head.
22장	LSC: 인프라 팀이 마이그레이션. 분할·자동화·추적.
23장	CI = 항상 출시 가능한 트렁크의 증명. presubmit/postsubmit.
24장	빌드 한 번·승격, 맥등 배포, 배포 ≠ 릴리즈, 릴리즈는 제품.
25장	컴퓨터의 유틸리티화. 선언적 스케줄링. 소유권 경계 재설계.

참고

1. Titus Winters, Tom Manshreck, Hyrum Wright. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media, 2020. — 구글 공식 무료 공개: abseil.io/resources/swe-book
2. 본 교습서는 원서의 내용을 한국어로 설명·재구성한 학습 자료이며, 원문의 복제가 아니다. 인용은 단문에 한정하고 출처를 표시했다.